

FSDP Seminar

Agenda

- CUDA Streams / Events
- ProcessGroup, Device Mesh, DTensor
- FSDP2: interface, internals
- Checkpoints: MetaDevice, PyTorch DCP, TorchFT

CUDA streams and events

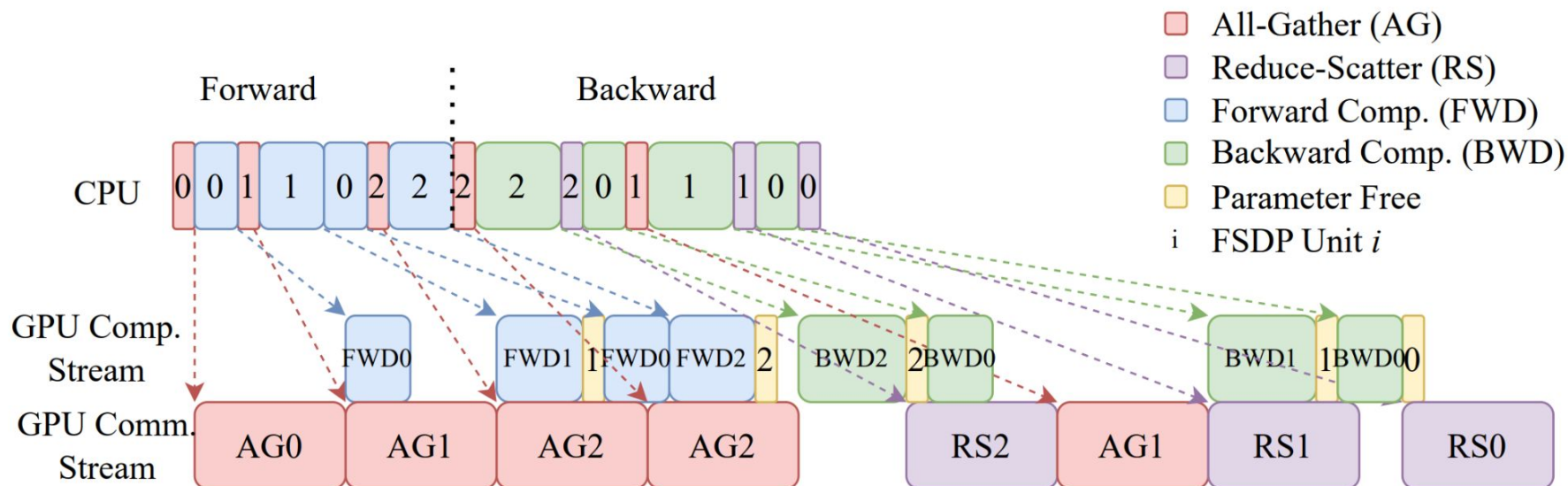


Figure 5: Overlap Communication and Computation

CUDA streams and events

```
all_gather_stream = torch.cuda.Stream()

...

# layer 3 unshard
with torch.cuda.stream(all_gather_stream):
    model.layers[3].all_gather()
    all_gather_event_3 = torch.cuda.Event()
    # or all_gather_stream.record_event()

# layer 2 forward
activations = model.layers[2](activations)

# layer 4 unshard
with torch.cuda.stream(all_gather_stream):
    model.layers[4].all_gather()
    all_gather_event_4 = torch.cuda.Event()

# layer 3 forward
torch.cuda.default_stream().wait_event(all_gather_event_3)
activations = model.layers[3](activations)

...
```

source: <https://docs.pytorch.org/docs/stable/generated/torch.cuda.Event.html>

source: <https://docs.pytorch.org/docs/stable/notes/cuda.html#asynchronous-execution>

CUDA streams and events



ProcessGroup

```
torch.distributed.init_process_group(backend=None, init_method=None,  
timeout=None, world_size=-1, rank=-1, store=None, group_name='',  
pg_options=None, device_id=None)
```

[\[source\]](#)

Initialize the default distributed process group.

This will also initialize the distributed package.

There are 2 main ways to initialize a process group:

1. Specify `store`, `rank`, and `world_size` explicitly.
2. Specify `init_method` (a URL string) which indicates where/how to discover peers.
Optionally specify `rank` and `world_size`, or encode all required parameters in the URL and omit them.

If neither is specified, `init_method` is assumed to be "env://".

ProcessGroup

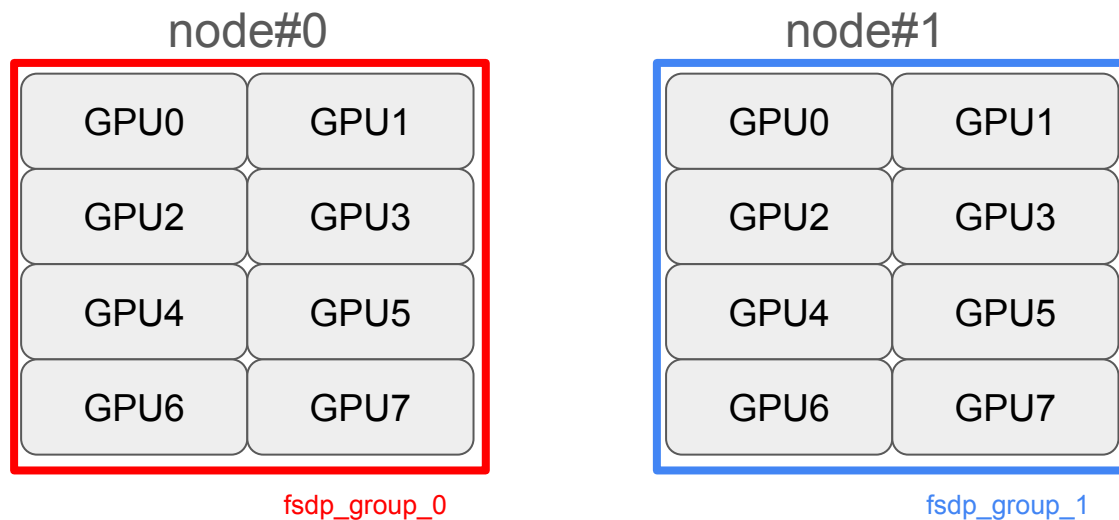
node#0

GPU0	GPU1
GPU2	GPU3
GPU4	GPU5
GPU6	GPU7

node#1

GPU0	GPU1
GPU2	GPU3
GPU4	GPU5
GPU6	GPU7

ProcessGroup



```
torch.distributed.new_group(ranks=None, timeout=None, backend=None,  
pg_options=None, use_local_synchronization=False, group_desc=None,  
device_id=None)
```

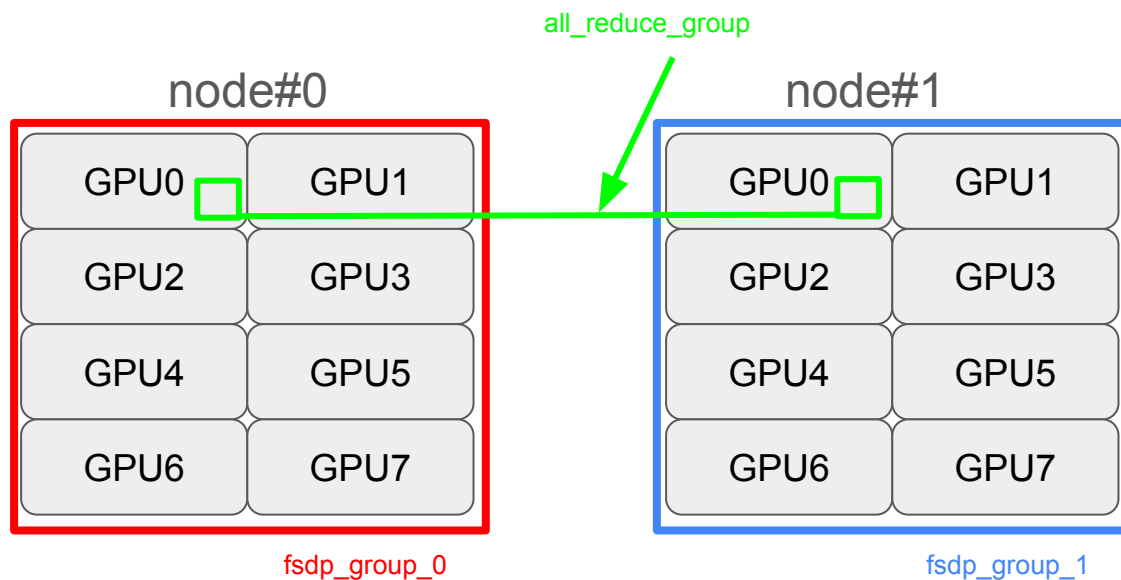
[\[source\]](#)

Create a new distributed group.

This function requires that all processes in the main group (i.e. all processes that are part of the distributed job) enter this function, even if they are not going to be members of the group.

Additionally, groups should be created in the same order in all processes.

ProcessGroup



```
torch.distributed.new_group(ranks=None, timeout=None, backend=None,  
pg_options=None, use_local_synchronization=False, group_desc=None,  
device_id=None)
```

[\[source\]](#)

Create a new distributed group.

This function requires that all processes in the main group (i.e. all processes that are part of the distributed job) enter this function, even if they are not going to be members of the group.

Additionally, groups should be created in the same order in all processes.

ProcessGroups manually

```
def _init_parallel_groups(self):
    """Initialize 3D parallelism's all process groups."""
    dist.barrier()
    ranks = np.arange(0, self.world_size).reshape(
        (
            self.expert_parallel_size,
            self.pipeline_parallel_size,
            self.data_parallel_size,
            self.context_parallel_size,
            self.tensor_parallel_size,
        )
    )
    self.world_ranks_to_pg = {}
    self.local_pg = self.create_new_group(ranks.reshape((-1, self.local_world_size)))
    assert int(os.environ.get("LOCAL_RANK")) == dist.get_rank(self.local_pg), "Local rank mismatch"

    # Relevant process groups containing the current rank
    self.tp_pg = self.create_new_group(ranks.transpose((0, 1, 2, 3, 4)).reshape((-1, self.tensor_parallel_size)))
    self.cp_pg = self.create_new_group(ranks.transpose((4, 0, 1, 2, 3)).reshape((-1, self.context_parallel_size)))
    self.dp_pg = self.create_new_group(ranks.transpose((3, 4, 0, 1, 2)).reshape((-1, self.data_parallel_size)))
    self.pp_pg = self.create_new_group(ranks.transpose((2, 3, 4, 0, 1)).reshape((-1, self.pipeline_parallel_size)))
    self.ep_pg = self.create_new_group(
        ranks.transpose((1, 2, 3, 4, 0)).reshape((-1, self.expert_parallel_size))
    ) # T000: ep should be a subset of dp

    # model parallel group = combination of tp and pp and exp for a given dp rank
    self.mp_pg = self.create_new_group(
        [
            ranks[:, :, dp_rank, cp_rank, :].reshape(-1)
            for cp_rank in range(self.context_parallel_size)
            for dp_rank in range(self.data_parallel_size)
        ]
    )

    self.dp_cp_pg = self.create_new_group(
        [
            ranks[ep_rank, pp_rank, :, :, tp_rank].reshape(-1)
            for tp_rank in range(self.tensor_parallel_size)
            for pp_rank in range(self.pipeline_parallel_size)
            for ep_rank in range(self.expert_parallel_size)
        ]
    )

    self.tp_and_ep_pg = self.create_new_group(
        [
            ranks[:, pp_rank, dp_rank, cp_rank, :].reshape(-1)
            for cp_rank in range(self.context_parallel_size)
            for pp_rank in range(self.pipeline_parallel_size)
            for dp_rank in range(self.data_parallel_size)
        ]
    )

    # self.tp_and_cp_pg = self.create_new_group(
    #     [
    #         ranks[ep_rank, pp_rank, dp_rank, :, :].reshape(-1)
    #         for ep_rank in range(self.expert_parallel_size)
    #         for pp_rank in range(self.pipeline_parallel_size)
    #         for dp_rank in range(self.data_parallel_size)
    #     ]
    # )

    self.world_rank_matrix: np.ndarray = ranks
    self.parallel_order = ["ep", "pp", "dp", "cp", "tp"]
```

```
def create_new_group(self, all_groups_ranks: np.ndarray) -> dist.ProcessGroup:
    dist.barrier()
    rank = int(os.environ["RANK"])
    new_group_containing_rank = None
    for group_ranks in all_groups_ranks:
        sorted_ranks = tuple(sorted(group_ranks))

        # add new group to `world_ranks_to_pg`
        if sorted_ranks not in self.world_ranks_to_pg:
            new_group = dist.new_group(ranks=group_ranks)
            self.world_ranks_to_pg[sorted_ranks] = new_group
        else:
            new_group = self.world_ranks_to_pg[sorted_ranks]

        if rank in sorted_ranks:
            new_group_containing_rank = new_group
    dist.barrier()
    return new_group_containing_rank
```

source: <https://github.com/huggingface/nanotron/blob/main/src/nanotron/parallel/context.py>

DeviceMesh

```
from torch.distributed.device_mesh import init_device_mesh

mesh_1d = init_device_mesh("cuda", mesh_shape=(8,), mesh_dim_names=("dp",))
mesh_2d = init_device_mesh("cpu", mesh_shape=(2, 8), mesh_dim_names=("dp_replicate", "dp_shard"))
mesh_3d = init_device_mesh(
    "cuda",
    mesh_shape=(2, 2, 8),
    mesh_dim_names=("pp", "dp", "tp"),
)

dp_group = mesh_2d.get_group("dp")
dist.all_gather(..., group=dp_group)

mesh_2d.get_local_rank("tp")

mesh_2d["dp_replicate", "dp_shard"]._flatten("dp")
```

DeviceMesh — from TorchTitan

```
def _build_mesh_without_ep(self) -> DeviceMesh:
    dims = []
    names = []
    for d, name in zip(
        [self.pp, self.dp_replicate, self.dp_shard, self.cp, self.tp],
        ["pp", "dp_replicate", "dp_shard", "cp", "tp"],
    ):
        if d > 1:
            dims.append(d)
            names.append(name)

    logger.info(f"Building {len(dims)}-D device mesh with {names}, {dims}")
    mesh = init_device_mesh(device_type, dims, mesh_dim_names=names)
```

DTensor

```
from torch.distributed.tensor import DTensor, distribute_tensor

mesh = init_device_mesh("cuda", mesh_shape=(8,), mesh_dim_names=("dp",))
big_tensor = torch.randn(1024, 4096)
placements = (Shard(dim=0),)

dtensor = distribute_tensor(
    big_tensor,
    device_mesh=mesh,
    placements=placements,
)
dtensor._local_tensor
dtensor.to_local() # .shape = (128, 4096)

shard = ... # .shape = (128, 4096)
DTensor.from_local(
    shard,
    device_mesh=mesh,
    placements=placements,
) # .shape = (1024, 4096)

dtensor.redistribute(placements=(Replicate(),))
dtensor.full_tensor()
```

DTensor

- `Shard`: Tensor sharded on the tensor dimension `dim` on the devices of the `DeviceMesh` dimension
- `Replicate`: Tensor replicated on the devices of the `DeviceMesh` dimension
- `Partial`: Tensor is pending reduction on the devices of the `DeviceMesh` dimension

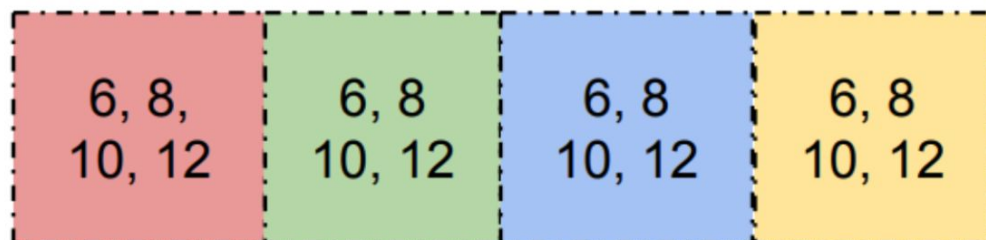
1. `Shard(dim) -> Replicate(): all_gather`
2. `Shard(src_dim) -> Shard(dst_dim): all_to_all`
3. `Replicate() -> Shard(dim): local chunking (i.e. torch.chunk)`
4. `Partial() -> Replicate(): all_reduce`
5. `Partial() -> Shard(dim): reduce_scatter`

DTensor

Data: [6, 8, 10, 12]



[Shard(0)]



[Replicate()]

DeviceMesh: ([0, 1, 2, 3])



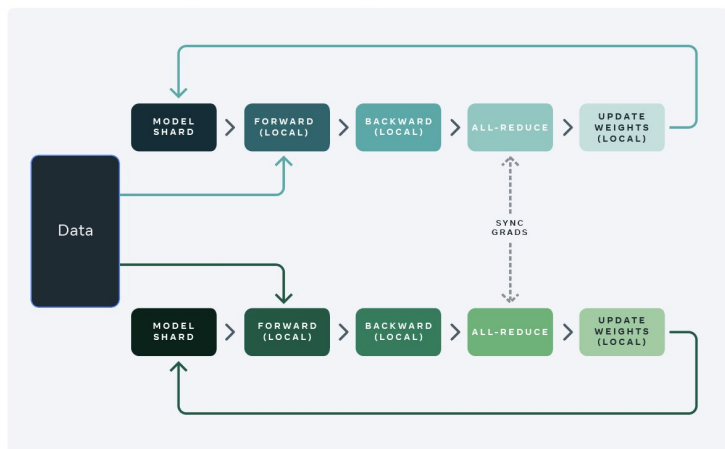
[Shard(0), Replicate()]

DeviceMesh: ([[0, 1], [2, 3]])

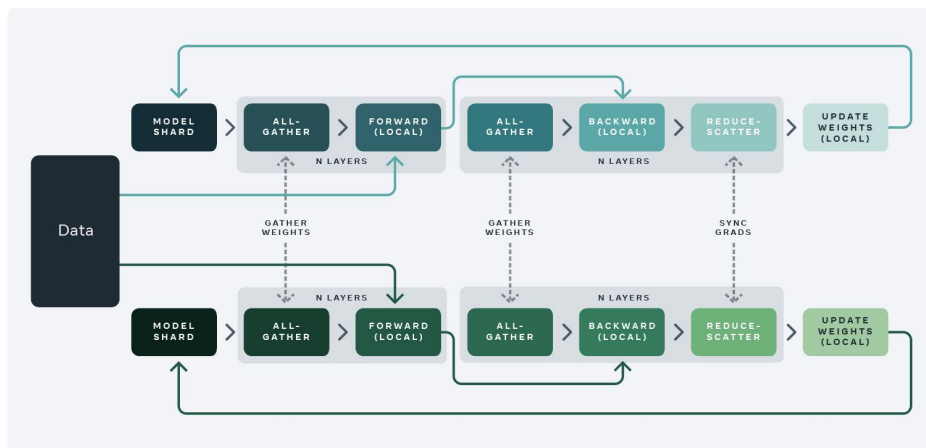
FSDP 2.0

FSDP 2.0

Standard data parallel training



Fully sharded data parallel training



FSDP 2.0

```
from torch.distributed.fsdp import fully_shard

mesh_2d = init_device_mesh(
    "cuda",
    mesh_shape=(2, 8),
    mesh_dim_names=("dp", "tp"),
)
model = Model()

for layer in model.layers:
    fully_shard(
        module, # (module1, module2)
        mesh=dp_mesh,
        reshard_after_forward=True, # ZeRO-3
        mp_policy=MixedPrecisionPolicy(
            param_dtype=torch.float16,
            reduce_dtype=torch.float32,
        ),
        offload_policy=CPUOffloadPolicy(),
    )

fully_shard(model, ...)
```

```
for step in ...:
    for gas_step in ...:
        is_last_backward = gas_step == num_gas_steps - 1
        # ZeRO-3
        model.set_reshard_after_backward(is_last_backward)
        # ZeRO-2
        model.requires_gradient_sync(is_last_backward)

    loss = loss_fn(model(inputs), targets)
    ...
```

FSDP 2.0 — Hooks

```
register_forward_pre_hook(hook, *, prepend=False, with_kwargs=False) \[SOURCE\]
```

Register a forward pre-hook on the module.

The hook will be called every time before `forward()` is invoked.

```
register_forward_hook(hook, *, prepend=False, with_kwargs=False, always_call=False) \[SOURCE\]
```

Register a forward hook on the module.

The hook will be called every time after `forward()` has computed an output.

FSDP 2.0 — pre-forward

```
def pre_forward(module, args):
    module.unshard() # in all-gather stream
    module.wait_for_unshard() # sync compute (default) stream with all-gather stream
    module._register_post_backward_hook(args)
    return args

def unshard(module):
    with torch.cuda.stream(all_gather_stream):
        module.all_gather()
    module.all_gather_event = all_gather_stream.record_event()
    module.set_unsharded_params()

def wait_for_unshard(module):
    torch.cuda.default_stream().wait_event(module.all_gather_event)

def fully_shard(module, ...):
    ...
    module.register_forward_pre_hook(pre_forward)
```

FSDP 2.0 — post-forward

```
def post_forward(module, args, output):
    module.reshard()
    module._record_post_forward()
    module._register_pre_backward_hook(output)
    return output

def reshard(module):
    module.set_sharded_params() # and free unsharded params

def _record_post_forward(module):
    post_forward_index = len(module.comm_ctx.post_forward_order)
    module.comm_ctx.post_forward_order.append(module)
    module._post_forward_indices.append(post_forward_index)

def fully_shard(module, ...):
    ...
    module.register_forward_hook(post_forward)
```

FSDP 2.0 — pre-backward

```
def pre_backward(module, *unused):
    module.unshard() # no-op if prefetched
    module.wait_for_unshard()
    module._backward_prefetch()

def _backward_prefetch(module):
    curr_index = module._post_forward_indices.pop()
    target_index = curr_index - 1
    target_module = self.comm_ctx.post_forward_order[target_index]
    target_module.unshard()

def _register_pre_backward_hook(self, output):
    for t in output:
        if torch.is_tensor(t) and t.requires_grad:
            t.register_hook(self._pre_backward)
    return output
```

FSDP 2.0 — post-backward

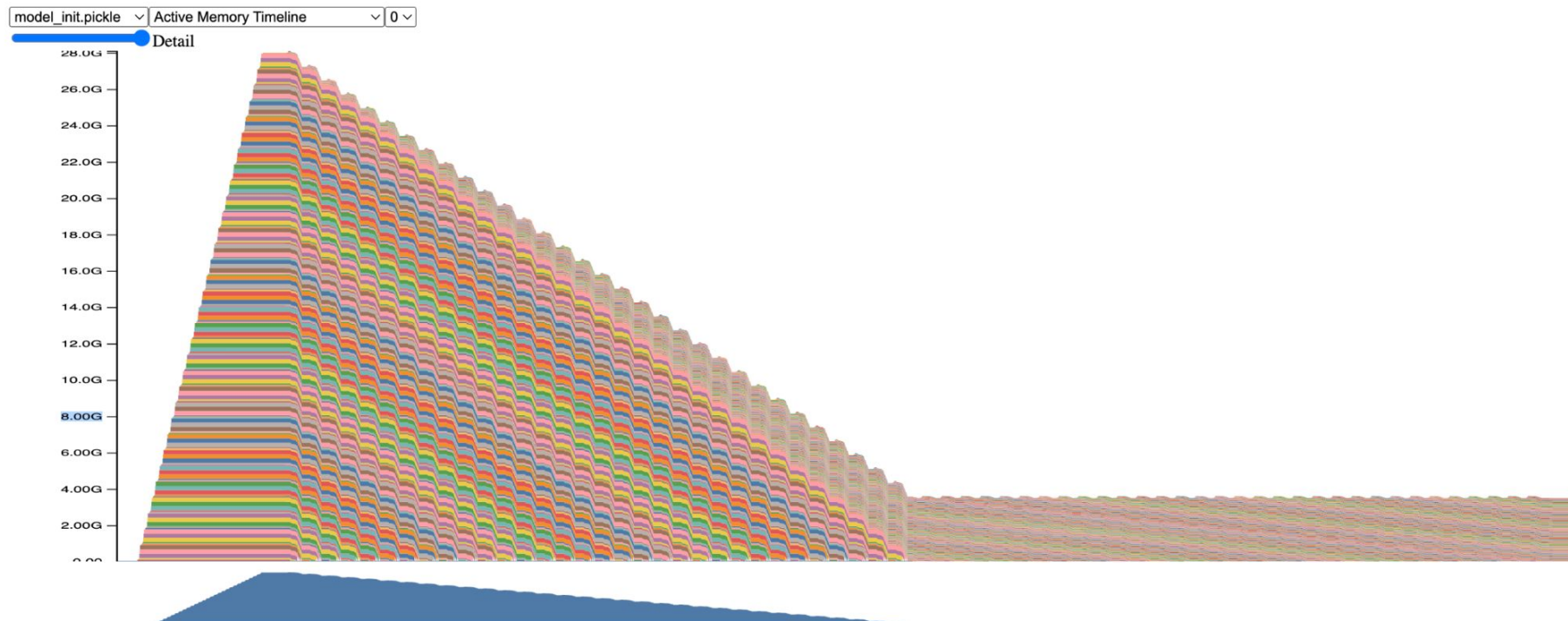
```
def post_backward(module, *unused: Any):
    if module.reshard_after_backward:
        module.reshard()
    if module.reduce_grads:
        reduce_scatter_stream.wait_stream(torch.cuda.default_stream())
        with torch.cuda.stream(reduce_scatter_stream):
            module.reduce_scatter_grads()
        reduce_event = reduce_scatter_stream.record_event()

def _register_post_backward_hook(module, args):
    RegisterPostBackwardFunction.apply(self, *args)

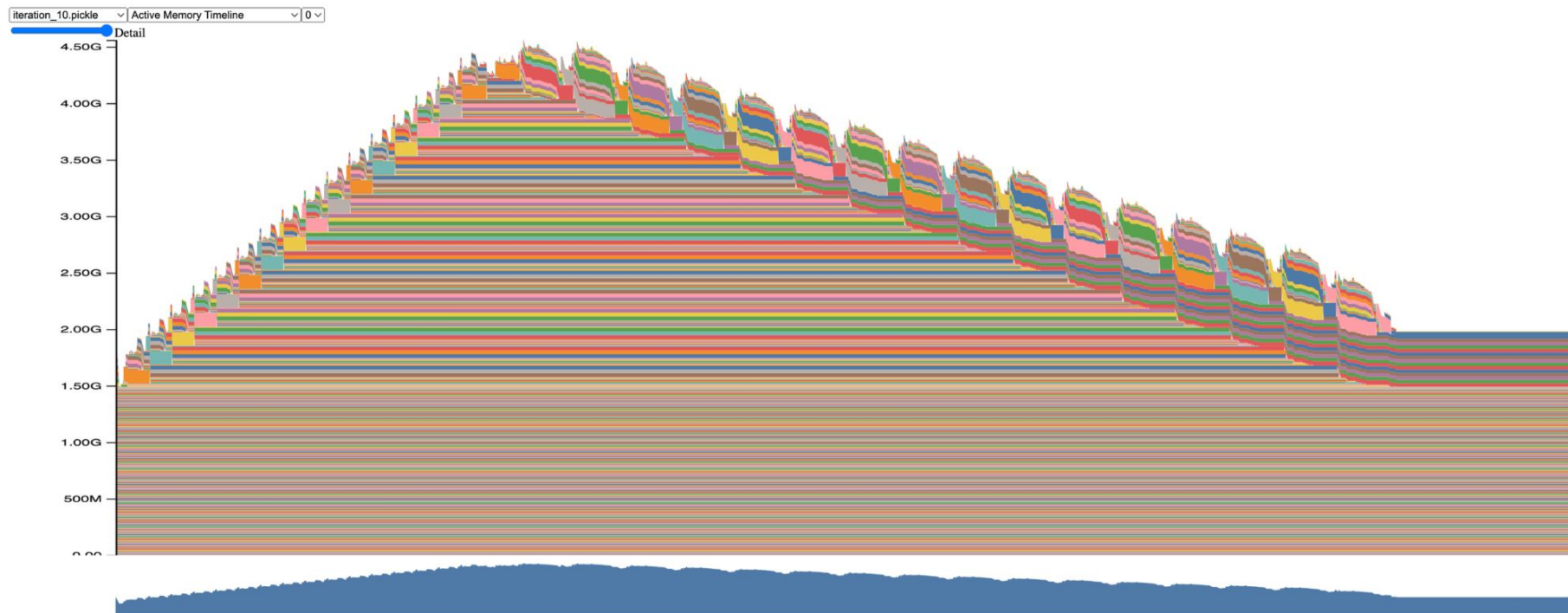
class RegisterPostBackwardFunction(torch.autograd.Function):
    @staticmethod
    def forward(ctx, module, *inputs):
        ctx.module = module
        return inputs

    @staticmethod
    def backward(ctx, *grads):
        module.post_backward()
        return (None,) + grads
```

FSDP 2.0 — init memory

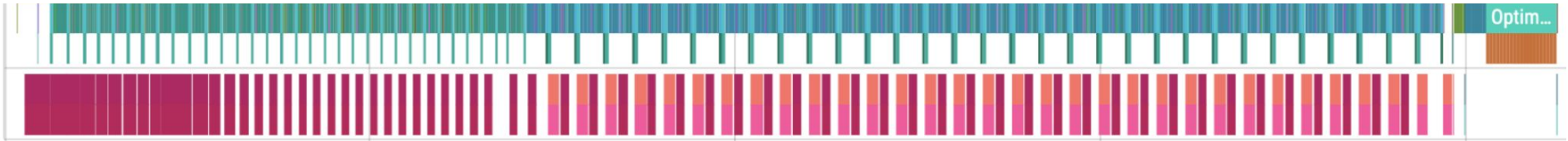


FSDP 2.0 — iteration memory



FSDP 2.0 — iteration profile

fwd+bwd+optim



fwd

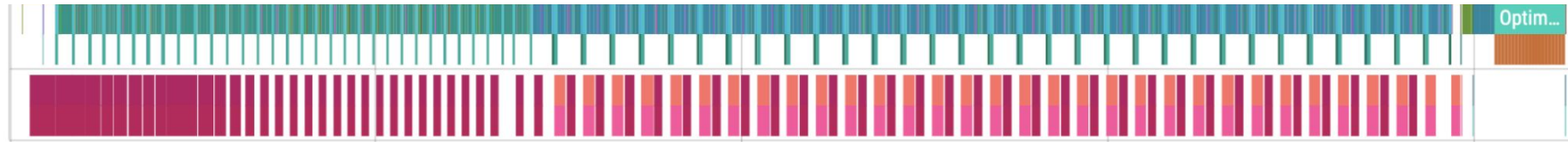


bwd

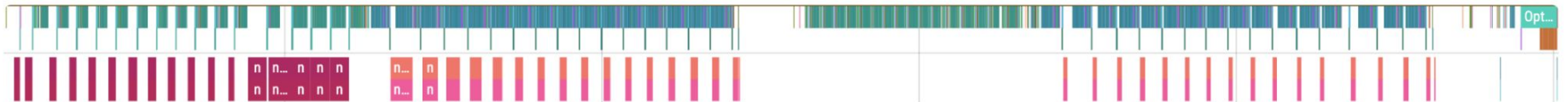


FSDP 2.0 — iteration profile

ZeRO-3



ZeRO-2



Checkpoints

Q

- How to init or load a large model?
- How to save checkpoints for sharded model?

MetaDevice

```
import torch.distributed.checkpoint as dcp
from torch.distributed.checkpoint.state_dict import get_state_dict, set_state_dict
from torch.distributed.checkpoint.stateful import Stateful

...

with torch.device("meta"):
    model = MyModel(config)

model = FSDP(
    model,
    device_id=torch.cuda.current_device(),
    param_init_fn=init_weights_if_needed,
)

app_state = {"model": model}

state_dict = get_state_dict(app_state)
dcp.load_state_dict(
    state_dict=state_dict,
    storage_reader=dcp.FileSystemReader(checkpoint_dir),
)
set_state_dict(app_state, state_dict)

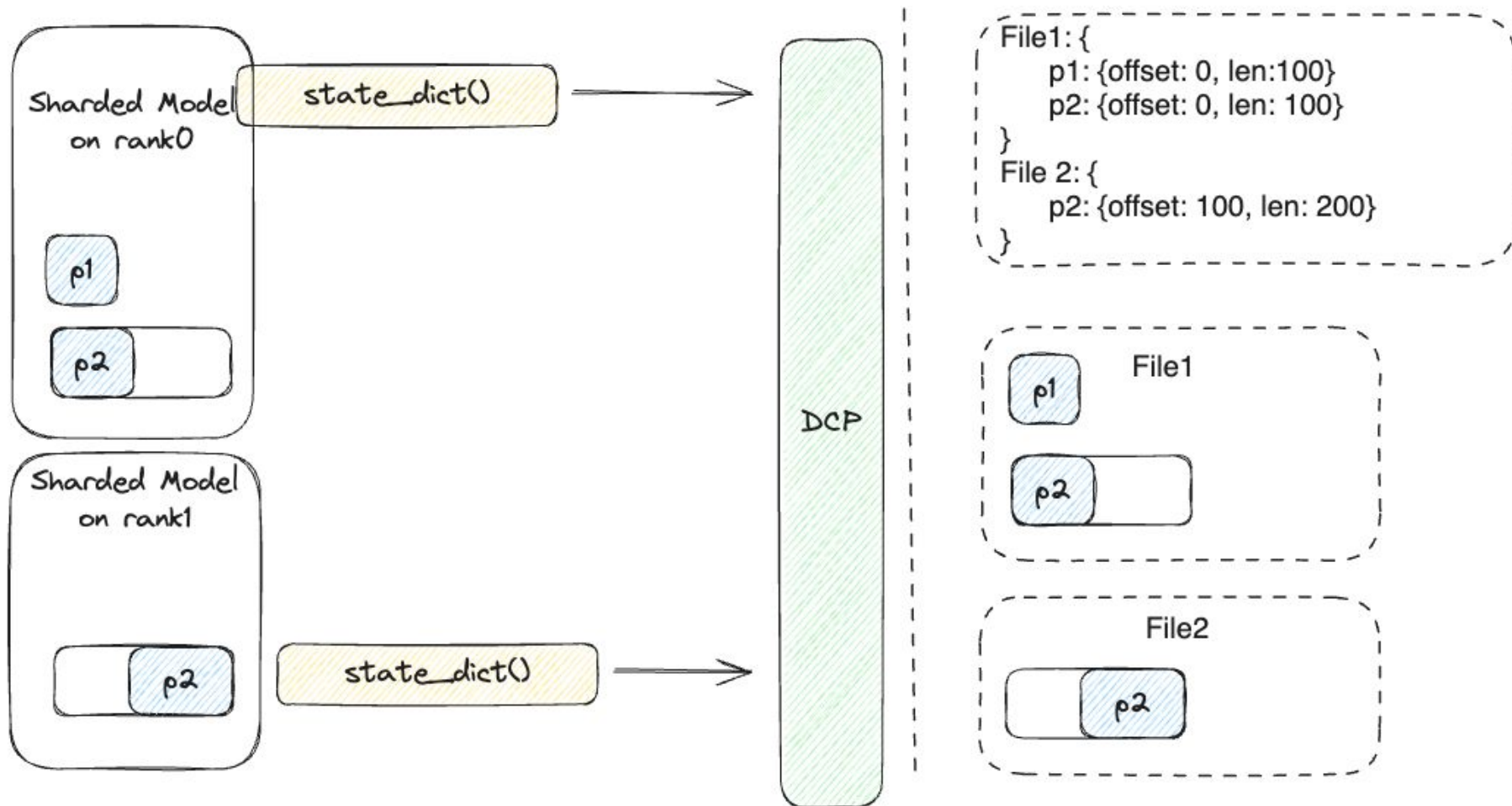
...
# trainer.train()
```

PyTorch DCP

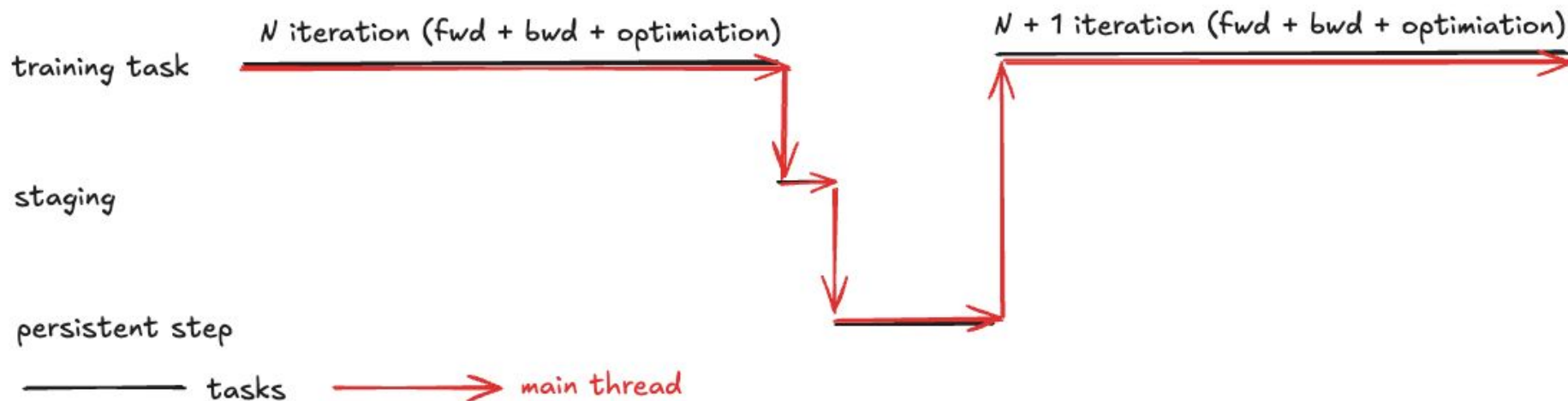
```
import torch.distributed.checkpoint as dcp
model = Model()
fully_shard(model)
optimizer = Optimizer(model.parameters())

state_dict = {
    "model": model.state_dict(),
    "optimizer": optimizer.state_dict()
}
dcp.state_dict_saver.save(state_dict)
dcp.state_dict_loader.load(state_dict)
```

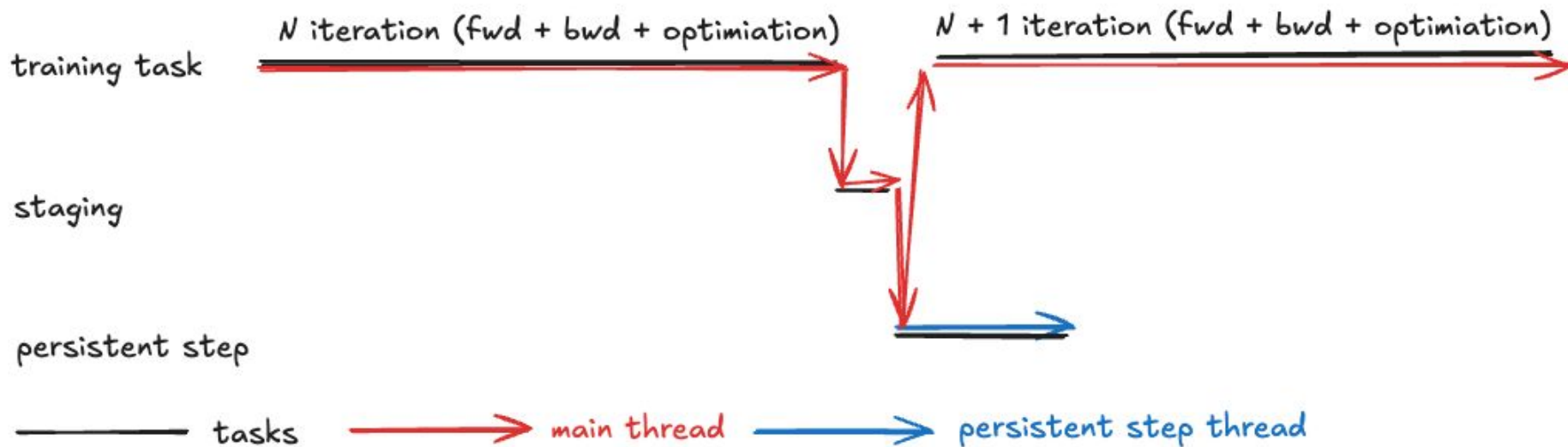
PyTorch DCP



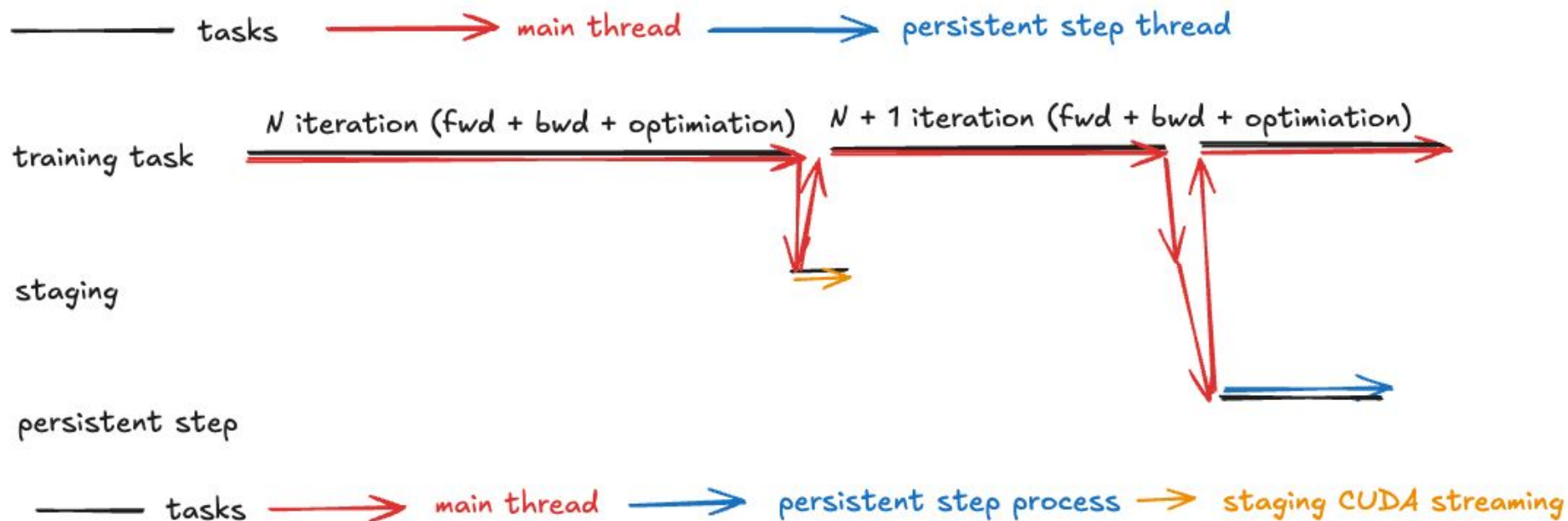
PyTorch DCP



PyTorch DCP



PyTorch DCP



HSDP

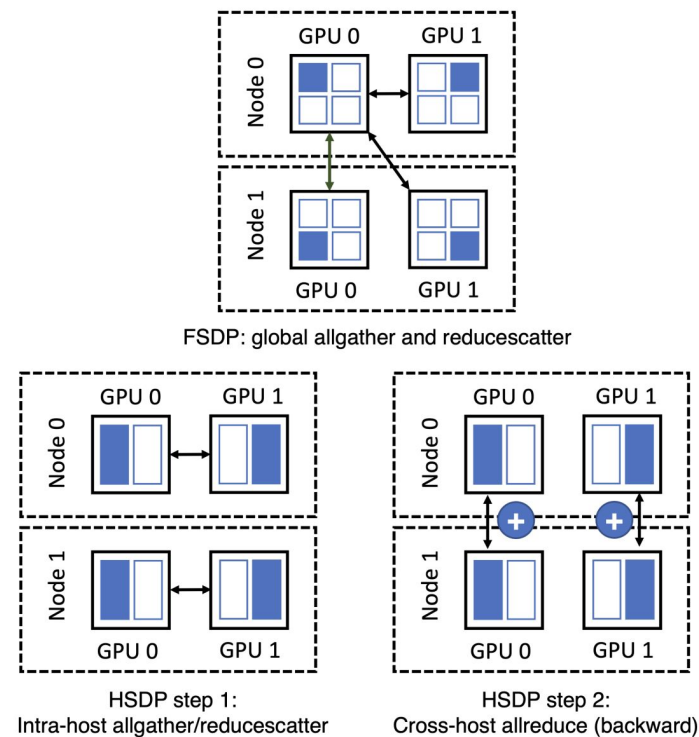
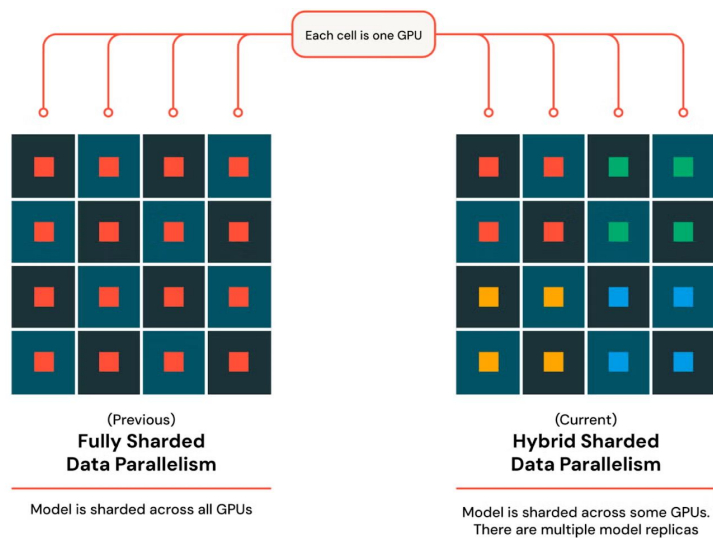
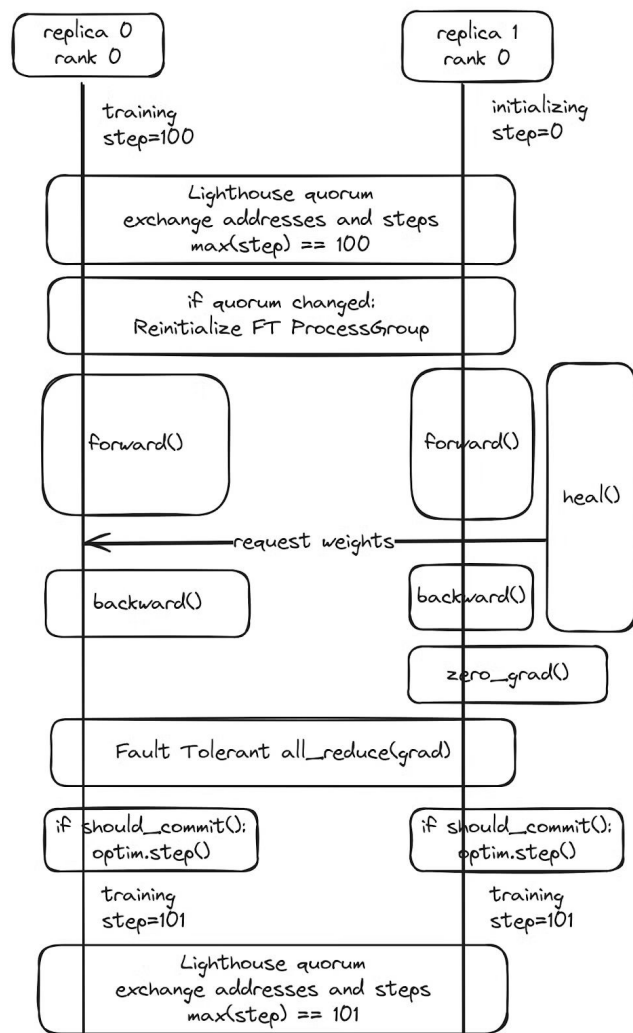


Figure 5: Comparison between FSDP (top) vs HSDP (bottom). Shown with 2 hosts with 2 GPUs each. We train large DHENs using FSDP, and medium-sized DHENs using HSDP.

HSDP

```
mesh_2d = init_device_mesh(  
    "cpu",  
    mesh_shape=(2, 8),  
    mesh_dim_names=("dp_replicate", "dp_shard"),  
)  
  
fully_shard(  
    module,  
    mesh=mesh_2d,  
    ...  
)
```

TorchFT



Extras

1. YaFSDP: <https://habr.com/ru/companies/yandex/articles/817509/>
2. SimpleFSDP: <https://arxiv.org/abs/2411.00284>
3. TorchTitan: <https://github.com/pytorch/torchtitan>